SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

I Mohamed Irfan.N (192521473) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)
A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥50 MB).
Evaluate how secure hash functions improve integrity compared to basic checksum methods.

---

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation
A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption).
Analyze the performance vs security trade-offs in TLS optimization.

---

3. Debugging and Optimizing Digital Signature Algorithm (DSA)
A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.
Fix errors in signature generation and verification steps.
Optimize modular arithmetic operations for faster execution.
Evaluate the impact of randomness quality on signature security.

---

4. Debugging Key Management System
A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).
Fix issues related to key lifecycle management (generation, distribution, rotation).
Optimize secure storage using encryption and access control mechanisms.
Analyze how poor key management can compromise an entire cryptographic system.

---

5. Optimization of Secure File Transfer Protocol (SFTP-like System)
A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.
Debug issues in encryption/decryption synchronization.
Fix errors in packet handling and integrity verification.
Optimize throughput for large file transfers (≥100 MB).
Compare the optimized system with insecure file transfer methods and justify the improvements.

ANSWERS

The following section presents the debugged and optimized solutions for each of the five tasks. For every task, the identified errors are listed first, followed by the buggy code snippet, the corrected/optimized code, and a short analytical evaluation, in line with the rubric criteria (Problem Identification, Debugging, Optimization, Analysis and Code Quality).

1. Debugging Hash Function Implementation (SHA Family)

1.1 Identified Errors

- Message padding did not append the mandatory single '1' bit (0x80 byte) before the zero padding, so the padded message length was incorrect.
- The 64-bit big-endian length field was appended using native byte order instead of big-endian, producing wrong length encoding on little-endian machines.
- The left-rotate function used a plain left shift (<<) instead of a circular rotate, discarding overflow bits and corrupting the compression function.
- Word values were not masked to 32 bits (& 0xFFFFFFFF) after addition, causing integer overflow since Python integers are arbitrary precision.
- The hash state variables (h0-h4) were re-initialized inside the block-processing loop instead of only once, so only the last block influenced the final digest — explaining the inconsistent output for the same input.

1.2 Buggy Code

```
import struct

def sha1_buggy(message: bytes) -> str:
    h0, h1, h2, h3, h4 = (0x67452301, 0xEFCDAB89,
                          0x98BADCFE, 0x10325476, 0xC3D2E1F0)

    ml = len(message) * 8
    message += b'\x80'                  # missing check, sometimes skipped
    while len(message) % 64 != 56:
        message += b'\x00'
    message += struct.pack('Q', ml)        # BUG: native byte order, not '>Q'

    for chunk_start in range(0, len(message), 64):
        # BUG: state re-initialised every block -> only last block counts
        h0, h1, h2, h3, h4 = (0x67452301, 0xEFCDAB89,
                              0x98BADCFE, 0x10325476, 0xC3D2E1F0)
        chunk = message[chunk_start:chunk_start + 64]
        w = list(struct.unpack('>16I', chunk)) + [0] * 64

        for i in range(16, 80):
            v = w[i-3] ^ w[i-8] ^ w[i-14] ^ w[i-16]
            w[i] = v << 1                # BUG: plain shift, not rotate

        a, b, c, d, e = h0, h1, h2, h3, h4
        for i in range(80):
            if i < 20:
                f, k = (b & c) | (~b & d), 0x5A827999
            elif i < 40:
                f, k = b ^ c ^ d, 0x6ED9EBA1
            elif i < 60:
                f, k = (b & c) | (b & d) | (c & d), 0x8F1BBCDC
            else:
                f, k = b ^ c ^ d, 0xCA62C1D6

            temp = a + f + e + k + w[i]      # BUG: no 32-bit mask
            e, d, c, b, a = d, c, b, a, temp

        h0, h1, h2, h3, h4 = h0+a, h1+b, h2+c, h3+d, h4+e

    return '%08x%08x%08x%08x%08x' % (h0, h1, h2, h3, h4)
```

1.3 Debugged and Optimized Code

```
import struct

MASK32 = 0xFFFFFFFF

def _rotl(n, b):
    return ((n << b) | (n >> (32 - b))) & MASK32

def sha1_fixed(message: bytes) -> str:
    h0, h1, h2, h3, h4 = (0x67452301, 0xEFCDAB89,
                          0x98BADCFE, 0x10325476, 0xC3D2E1F0)

    ml = len(message) * 8
    padded = message + b'\x80'
    while len(padded) % 64 != 56:
        padded += b'\x00'
    padded += struct.pack('>Q', ml)          # FIX: explicit big-endian

    # FIX: state persists across blocks, initialised only once above
    for chunk_start in range(0, len(padded), 64):
        chunk = padded[chunk_start:chunk_start + 64]
        w = list(struct.unpack('>16I', chunk)) + [0] * 64

        for i in range(16, 80):
            v = w[i-3] ^ w[i-8] ^ w[i-14] ^ w[i-16]
            w[i] = _rotl(v, 1)               # FIX: true circular rotate

        a, b, c, d, e = h0, h1, h2, h3, h4
        for i in range(80):
            if i < 20:
                f, k = (b & c) | (~b & d), 0x5A827999
            elif i < 40:
                f, k = b ^ c ^ d, 0x6ED9EBA1
            elif i < 60:
                f, k = (b & c) | (b & d) | (c & d), 0x8F1BBCDC
            else:
                f, k = b ^ c ^ d, 0xCA62C1D6

            temp = (_rotl(a, 5) + f + e + k + w[i]) & MASK32   # FIX: masked
            e, d, c, b, a = d, c, b, a, temp

        h0 = (h0 + a) & MASK32
        h1 = (h1 + b) & MASK32
        h2 = (h2 + c) & MASK32
        h3 = (h3 + d) & MASK32
        h4 = (h4 + e) & MASK32

    return '%08x%08x%08x%08x%08x' % (h0, h1, h2, h3, h4)


def sha1_large_file(path: str, buf_size: int = 8 * 1024 * 1024) -> str:
    """Stream a >=50 MB file in fixed-size chunks instead of loading it
    fully into memory, cutting peak RAM usage and I/O wait time."""
    import hashlib
    h = hashlib.sha1()
    with open(path, 'rb') as f:
        while chunk := f.read(buf_size):
            h.update(chunk)
    return h.hexdigest()
```

1.4 Optimization Notes

For files at or above 50 MB the pure-Python bit-level implementation above is kept only for teaching the internal mechanics; production hashing uses hashlib (a C-optimised SHA implementation) together with buffered streaming reads (8 MB chunks) instead of message = open(path).read(), which

avoids loading the entire file into memory and reduces wall-clock time on large files from minutes to a few seconds.
The 8 MB buffer size balances system-call overhead against memory footprint; smaller buffers increase the number of read() calls, larger buffers increase peak memory without a proportional speed gain.

1.5 Evaluation: Hashing vs Checksums

A checksum such as CRC32 is designed only to catch accidental transmission errors; it is linear and trivially invertible, so an attacker can modify data and recompute a matching checksum in constant time.
A cryptographic hash such as SHA-1/SHA-256 is designed to be pre-image resistant, second-preimage resistant and collision resistant, so even a single-bit change in the input produces an unpredictable, avalanche-style change in the digest, making undetected tampering computationally infeasible.
Consequently, SHA-family hashes are suitable for integrity verification in security-sensitive contexts (digital signatures, software distribution, blockchain), whereas checksums remain adequate only for detecting non-malicious transmission noise.

---

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

2.1 Identified Errors

- Certificate expiry (notAfter) and issuer chain were never checked, so an expired or self-signed rogue certificate was silently accepted.
- The Diffie-Hellman key exchange re-generated fresh (g, p) parameters on every single connection instead of using vetted, cached parameters, which is both slow and a known source of handshake failures when weak primes are generated.
- The shared secret was recomputed from scratch for every request within the same session instead of being cached, causing unnecessary latency on repeat connections.
- There was no session-ID / session-ticket mechanism, so every reconnect performed a full handshake even for the same client, defeating TLS session resumption.

2.2 Buggy Code

```
class TLSHandshakeBuggy:
    def     init    (self):
        self.sessions = {}

    def validate_certificate(self, cert):
        # BUG: only checks the subject name, ignores expiry & issuer chain
        return cert.get("subject") is not None

    def key_exchange(self, client_id):
        # BUG: fresh DH parameters generated every call -> slow & unsafe
        p, g = generate_dh_parameters()
        priv = random_priv_key()
        pub = pow(g, priv, p)
```

```
        shared_secret = pow(pub, priv, p)   # BUG: not combined with peer key
        return shared_secret

    def handshake(self, client_id, cert):
        if not self.validate_certificate(cert):
            raise Exception("invalid cert")
        # BUG: no session cache -> full handshake every single time
        secret = self.key_exchange(client_id)
        return secret
```

2.3 Debugged and Optimized Code

```
import time

# FIX: DH parameters generated once and reused (vetted RFC 3526 group)
DH_P, DH_G = load_standard_dh_group()

class TLSHandshakeFixed:
    def __init__(self, session_ttl=3600):
        self.session_cache = {} # session_id -> (secret, expiry) self.session_ttl
        = session_ttl

    def validate_certificate(self, cert, now=None):
        now = now or time.time() if
        cert.get("subject") is None:
            return False # FIX: expiry
        check if cert.get("not_after", 0)
        < now:
            return False
        # FIX: issuer chain verified against trusted root store if
        not verify_chain(cert, TRUSTED_ROOTS):
            return False
        return True

    def key_exchange(self, client_pub_key):
        priv = random_priv_key()
        my_pub = pow(DH_G, priv, DH_P)
        # FIX: shared secret derived from BOTH keys
        shared_secret = pow(client_pub_key, priv, DH_P)
        return my_pub, shared_secret

    def handshake(self, client_id, cert, client_pub_key): if
        not self.validate_certificate(cert):
            raise ValueError("certificate validation failed")

        # FIX: session resumption avoids a full handshake on reconnect
        cached = self.session_cache.get(client_id) if cached and
        cached[1] > time.time(): return cached[0]

        my_pub, secret = self.key_exchange(client_pub_key)
        self.session_cache[client_id] = (secret, time.time() + self.session_ttl)
        return secret
```

2.4 Performance vs Security Trade-off

Reusing a vetted DH group removes the cost of prime generation (which dominates handshake latency) at the cost of relying on a small number of standardised, publicly known groups; this is the same trade-off TLS 1.3 makes by restricting key exchange to a fixed list of named groups. Session resumption (session IDs/tickets) reduces a full handshake to a single round trip, cutting latency substantially, but it also extends the lifetime of a compromised key beyond a single connection, which is why resumed sessions are given a bounded TTL (here, one hour) and

forwardsecrecy is preserved by deriving fresh traffic keys per session rather than reusing the master secret directly.
Overall the optimisation trades a small, well-understood exposure window for a large latency reduction, which matches the accepted engineering practice used in production TLS stacks.

---

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

3.1 Identified Errors

- The per-signature random nonce k was generated with Python's non-cryptographic random module and, worse, was fixed to a constant value for testing and never changed back — reusing k across signatures leaks the private key.
- The signature verification step recomputed w = pow(s, -1, q) using integer '**' instead of modular inverse, so it silently produced a wrong value instead of raising an error.
- Parameter selection did not verify that q divides (p-1), so invalid (p, q, g) groups could be accepted.
- Modular exponentiation used the built-in ** operator followed by % q on very large numbers, computing the full power before reducing, which is far slower than Python's 3-argument pow(base, exp, mod).

3.2 Buggy Code

```
import random

def sign_buggy(msg_hash, x, p, q, g):
   k = 12345 # BUG: constant nonce, reused every call r = pow(g, k,
   p) % q s = (pow(k, -1, q) * (msg_hash + x * r)) % q return r, s

def verify_buggy(msg_hash, r, s, p, q, g, y):
   if not (0 < r < q and 0 < s < q):
      return False
   w = s ** -1 % q                # BUG: '**' with negative exponent on
                           # an int does NOT compute modular
                           # inverse in Python -> raises/garbage
   u1 = (msg_hash * w) % q
   u2 = (r * w) % q v = ((pow(g, u1, p) * pow(y,
   u2, p)) % p) % q return v == r
```

3.3 Debugged and Optimized Code

```
import secrets

def sign_fixed(msg_hash, x, p, q, g):
    while True:
        k = secrets.randbelow(q - 1) + 1     # FIX: fresh CSPRNG nonce
        r = pow(g, k, p) % q
        if r == 0:
            continue
        k_inv = pow(k, -1, q)                # FIX: correct modular inverse
        s = (k_inv * (msg_hash + x * r)) % q
        if s != 0:
            return r, s

def verify_fixed(msg_hash, r, s, p, q, g, y):
    if not (0 < r < q and 0 < s < q):
        return False
    w = pow(s, -1, q)                        # FIX: pow(base, -1, mod)
    u1 = (msg_hash * w) % q
    u2 = (r * w) % q
    # FIX: 3-argument pow keeps every intermediate value bounded by p,
    # instead of computing astronomically large integers first
    v = ((pow(g, u1, p) * pow(y, u2, p)) % p) % q
    return v == r

def validate_domain_parameters(p, q, g):
    # FIX: reject malformed groups before they are ever used
    if (p - 1) % q != 0:
        raise ValueError("q does not divide p-1")
    if pow(g, q, p) != 1:
        raise ValueError("g does not have order q")
    return True
```

3.4 Evaluation: Impact of Randomness Quality

DSA security depends entirely on the nonce k being unique, uniformly random and secret for every signature. Reusing k across two signatures on different messages, or generating it with a predictable PRNG, allows an attacker to solve two linear equations in the two unknowns (x, k) and recover the private key directly (as happened in the well-known Sony PS3 and several Bitcoin wallet keyrecovery incidents).

Replacing random with secrets.randbelow ensures the nonce is drawn from a cryptographically secure source, eliminating this entire class of key-recovery attack, which is why the fix is classified as a critical security correction rather than a stylistic improvement.

---

4. Debugging Key Management System

4.1 Identified Errors

- Private keys were written to disk as plaintext PEM files inside the project folder with default file permissions, so any local process or user could read them.
- There was no key rotation logic: a single key pair was generated once and reused indefinitely, increasing the impact window of any future compromise.
- Key retrieval performed no access-control or authentication check before returning the key material.
- Old/expired keys were never revoked or archived separately, so compromised keys stayed usable.

4.2 Buggy Code

```
import os

class KeyStoreBuggy:
    def     init    (self, path="keys/"):
        self.path = path

    def store_key(self, key_id, key_bytes):
        # BUG: plaintext storage, no encryption, no access control
        with open(os.path.join(self.path, key_id), "wb") as f:
            f.write(key_bytes)

    def get_key(self, key_id):
        # BUG: no authentication / authorisation check
        with open(os.path.join(self.path, key_id), "rb") as f:
            return f.read()

    def generate_key(self, key_id):
        key = os.urandom(32)
         self.store_key(key_id, key)
         return key
         # BUG: no rotation schedule, no expiry metadata recorded
```

4.3 Debugged and Optimized Code

```
import    os
import
time
import json
from cryptography.fernet import Fernet

class KeyStoreFixed:
   """Keys are encrypted at rest under a master key (KEK) that itself
   should be held in an HSM / OS key vault, not on the same disk."""

   def _init_(self, path="keys/", master_key: bytes = None, rotation_days:
            int = 90):
      self.path = path
      os.makedirs(self.path,   exist_ok=True,   mode=0o700)
      self.master_key = master_key or Fernet.generate_key()
      self.fernet = Fernet(self.master_key) self.rotation_days
      = rotation_days
      self.metadata_file = os.path.join(self.path, "metadata.json")
      self._load_metadata()

   def _load_metadata(self):
      if os.path.exists(self.metadata_file):
         with open(self.metadata_file) as f:
            self.metadata = json.load(f)
      else: self.metadata =
         {}

   def _save_metadata(self):
      with open(self.metadata_file, "w") as f: json.dump(self.metadata,
         f)

   def generate_key(self, key_id, requester_role="admin"):
      self._authorize(requester_role) # FIX: access control key =
      os.urandom(32) encrypted = self.fernet.encrypt(key) # FIX:
      encrypt at rest key_path = os.path.join(self.path, key_id) with
      open(key_path, "wb") as f:
         f.write(encrypted)
      os.chmod(key_path,      0o600)      #      FIX:      restrict      perms
      self.metadata[key_id] = {
         "created": time.time(),
         "rotate_after": time.time() + self.rotation_days * 86400, "status":
         "active",
      }
      self._save_metadata()
      return key

   def get_key(self, key_id, requester_role="admin"):
      self._authorize(requester_role) # FIX: enforce ACL meta =
      self.metadata.get(key_id) if not meta or meta["status"] !=
      "active":
         raise PermissionError("key not available")
      if time.time() > meta["rotate_after"]:              # FIX: rotation
         self.rotate_key(key_id, requester_role) meta =
         self.metadata[key_id]
      with open(os.path.join(self.path, key_id), "rb") as f: return
         self.fernet.decrypt(f.read())

   def rotate_key(self, key_id, requester_role="admin"):
      self.revoke_key(key_id, requester_role)
      return self.generate_key(key_id + "_v2", requester_role)
```

```
def revoke_key(self, key_id, requester_role="admin"):
    self._authorize(requester_role)
    if key_id in self.metadata:
        self.metadata[key_id]["status"] = "revoked"       # FIX: revocation
        self._save_metadata()

def _authorize(self, role):
    if role not in ("admin", "service"):
        raise PermissionError("unauthorized role")
```

4.4 Evaluation

A key management system is the trust anchor of the entire cryptographic stack: if keys can be read by unauthorised processes, every algorithm built on top of them (AES, RSA, DSA, TLS) is rendered meaningless, regardless of how mathematically strong the algorithm itself is.
Encrypting keys at rest under a separate master key, enforcing role-based access control, restricting file permissions, and rotating keys on a fixed schedule together shrink both the likelihood of exposure and the blast radius if a single key is ever compromised, which is the standard defence-in-depth approach used in production key management services (e.g. AWS KMS, HashiCorp Vault).

---

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

5.1 Identified Errors

- The encryption routine used a freshly generated IV for AES-CBC but never transmitted it to the receiver, so decryption used a mismatched IV and produced a corrupted first block on every transfer.
- Data was encrypted and sent as a single large in-memory blob, so throughput dropped sharply and memory usage spiked for files at or above 100 MB.
- No integrity check (MAC/HMAC) was performed on received packets, so silent corruption or tampering during transfer went undetected until the file failed to open.
- Packets were sent without sequence numbers, so out-of-order arrival on the receiving side silently produced a shuffled, corrupted file.

5.2 Buggy Code

```
from Crypto.Cipher import AES import
os

def send_file_buggy(path, sock, key):
    with open(path, "rb") as f:
        data = f.read()                  # BUG: whole file loaded at once
    iv = os.urandom(16)
    cipher = AES.new(key, AES.MODE_CBC, iv) padded
    = pad(data)
    ciphertext = cipher.encrypt(padded)
    sock.send(ciphertext)                # BUG: IV never sent to peer
                                         # BUG: no MAC, no sequence number

def recv_file_buggy(sock, key, iv, out_path):
    ciphertext = sock.recv(1 << 30)
    cipher = AES.new(key, AES.MODE_CBC, iv) # BUG: iv is a guess, not the
                                            # one actually used to encrypt
    data = unpad(cipher.decrypt(ciphertext)) with
    open(out_path, "wb") as f:
        f.write(data)
```

5.3 Debugged and Optimized Code

```
from Crypto.Cipher import AES from
Crypto.Hash import HMAC, SHA256
import os import struct

CHUNK_SIZE = 4 * 1024 * 1024 # 4 MB per packet

def send_file_fixed(path, sock, enc_key, mac_key):
   seq = 0 with open(path,
   "rb") as f:
      while chunk := f.read(CHUNK_SIZE):          # FIX: streamed, not
                                          # loaded fully in memory
         iv = os.urandom(16)
         cipher = AES.new(enc_key, AES.MODE_CBC, iv) padded
         = pad(chunk)
         ciphertext = cipher.encrypt(padded)

         mac = HMAC.new(mac_key, digestmod=SHA256)
         mac.update(struct.pack(">I", seq) + iv + ciphertext)
         tag = mac.digest()                    # FIX: integrity check

         # FIX: sequence number + IV are sent alongside the ciphertext
         packet = struct.pack(">I", seq) + iv + tag + ciphertext
         sock.sendall(struct.pack(">I", len(packet)) + packet) seq += 1
sock.sendall(struct.pack(">I", 0))                # end-of-stream marker


def recv_file_fixed(sock, enc_key, mac_key, out_path):
   expected_seq = 0 with
   open(out_path, "wb") as out:
      while True:
         length = struct.unpack(">I", _recv_exact(sock, 4))[0] if
         length == 0:
            break
         packet = _recv_exact(sock, length) seq
         = struct.unpack(">I", packet[:4])[0]
         iv, tag, ciphertext = packet[4:20], packet[20:52], packet[52:]

         mac = HMAC.new(mac_key, digestmod=SHA256)
         mac.update(packet[:4] + iv + ciphertext)
         mac.verify(tag)                    # FIX: rejects tampering

         if seq != expected_seq:             # FIX: ordering check raise
            ValueError(f"out-of-order packet: {seq}")
         expected_seq += 1

         cipher = AES.new(enc_key, AES.MODE_CBC, iv) # FIX: real IV used
         out.write(unpad(cipher.decrypt(ciphertext)))


def _recv_exact(sock, n):
   buf = b"" while
   len(buf) < n:
      part = sock.recv(n - len(buf)) if
      not part:
         raise ConnectionError("connection closed early")
      buf += part return
   buf
```

5.4 Comparison with Insecure File Transfer

Plain FTP transmits both credentials and file contents unencrypted, so any network observer can read or modify data in transit, and corruption is only detectable if the application happens to compare checksums out of band.

The optimized implementation above adds three properties plain FTP lacks: confidentiality (AESCBC with a unique IV per chunk), integrity/authenticity (HMAC-SHA256 per packet), and ordered, streamed delivery (sequence numbers plus fixed-size chunking), while the chunked streaming design keeps memory usage constant regardless of file size and allows overlapping I/O and encryption, which restores throughput that a single monolithic encrypt-then-send call would otherwise lose on files at or above 100 MB.
The justified overhead is the extra 4 bytes (sequence) + 16 bytes (IV) + 32 bytes (HMAC tag) per 4 MB chunk — under 0.002% bandwidth cost — for a system that is now resistant to eavesdropping, tampering and replay, which is a favourable trade-off for any file transfer carrying sensitive data.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, wellstructured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation